

# rgbCTF 2020 Crypto - Adequate Encryption Standard

deuterium — 2020-07-13

Canonical HTML: [https://deuterium.github.io/WriteUps/2020/rgbctf/crypto/Adequate Encryption Standard/2020-07-13-rgbctf-2020-Adequate-Encryption-standard.html](https://deuterium.github.io/WriteUps/2020/rgbctf/crypto/Adequate%20Encryption%20Standard/2020-07-13-rgbctf-2020-Adequate-Encryption-standard.html)

## Abstract

CTF writeup: rgbCTF 2020's Adequate Encryption Standard has a custom cipher whose key schedule gets stuck on 0 or 1. Reverse the rounds and exploit the rut.

## Aedquate Encryption Standard

### Description

I wrote my own AES! Can you break it?

hQWYogqLXU0+rePyWkNlBlaAX47/2dCeLFMLrmPKcYRLYZgFuqRC7EtwX4DRtG31XY4az+y0vJJ/pwWR0/J9gg  
==

~qpwoeirut#5057

### Files

- [adequate encryption standard.py](#)

```
from base64 import b64encode
```

```
BLOCK_SIZE = 8  
ROUNDS = 8
```

```
sbox = [111, 161, 71, 136, 68, 69, 31, 0, 145, 237, 169, 115, 16, 20, 22, 82, 138,  
183, 232, 95, 244, 163, 64, 229, 224, 104, 231, 61, 121, 152, 97, 50, 74, 96, 247,  
144, 194, 86, 186, 234, 99, 122, 46, 18, 215, 168, 173, 188, 41, 243, 219, 203, 141,  
21, 171, 57, 116, 178, 233, 210, 184, 253, 151, 48, 206, 250, 133, 44, 59, 147, 137,  
66, 52, 75, 187, 129, 225, 209, 191, 92, 238, 127, 241, 25, 160, 9, 170, 13, 157, 45,  
205, 196, 28, 146, 142, 150, 17, 39, 24, 80, 118, 6, 32, 93, 11, 216, 220, 100, 85,  
112, 222, 226, 126, 197, 180, 34, 182, 37, 148, 70, 78, 201, 236, 81, 62, 42, 193, 67,  
8, 164, 43, 252, 166, 221, 208, 176, 235, 149, 109, 63, 103, 223, 65, 56, 140, 255,  
218, 54, 153, 2, 228, 1, 240, 248, 246, 110, 156, 60, 227, 207, 254, 51, 174, 79,  
128, 155, 251, 242, 177, 135, 230, 154, 179, 15, 189, 143, 130, 27, 107, 211, 30, 105,  
19, 134, 124, 125, 245, 76, 204, 12, 26, 38, 40, 131, 117, 87, 114, 213, 212, 102,  
195, 101, 55, 10, 47, 120, 200, 217, 88, 83, 36, 198, 249, 192, 23, 94, 181, 73, 185,  
172, 165, 58, 53, 202, 106, 5, 7, 175, 89, 72, 90, 14, 162, 158, 119, 139, 77, 108,  
190, 91, 29, 49, 159, 33, 113, 214, 4, 123, 199, 167, 35, 239, 84, 3, 132, 98]  
pbox = [39, 20, 18, 62, 4, 60, 19, 43, 33, 6, 51, 61, 40, 35, 47, 16, 23, 58, 31, 53,  
28, 55, 54, 30, 17, 42, 34, 45, 49, 13, 46, 0, 26, 2, 8, 3, 11, 48, 63, 36, 37, 7, 32,  
5, 27, 59, 29, 44, 14, 56, 21, 22, 12, 52, 57, 41, 10, 1, 24, 38, 50, 15, 9, 25]
```

```

def pad(block):
    return block + chr(BLOCK_SIZE - len(block)).encode() * (BLOCK_SIZE - len(block))

def to_blocks(in_bytes: bytes) -> list:
    return [in_bytes[i:i + BLOCK_SIZE] for i in range(0, len(in_bytes), BLOCK_SIZE)]

def enc_sub(in_bytes: bytes) -> bytes:
    return bytes([sbox[b] for b in in_bytes])

def enc_perm(in_bytes: bytes) -> bytes:
    num = int.from_bytes(in_bytes, 'big')
    binary = bin(num)[2:].rjust(BLOCK_SIZE * 8, '0')
    permuted = ''.join([binary[pbox[i]] for i in range(BLOCK_SIZE * 8)])
    out = bytes([int(permuted[i:i + 8], 2) for i in range(0, BLOCK_SIZE * 8, 8)])
    return out

def expand_key(key: bytes, key_len: int) -> bytes:
    expanded = bytearray()
    cur = 0
    for byte in key:
        cur = (cur + byte) & ((1 << 8) - 1)
    expanded.append(cur)
    for num in [key[i % len(key)] * 2 for i in range(key_len)]:
        cur = pow(cur, num, 256)
        expanded.append(cur)
    return bytes(expanded)

def encrypt(plain: bytes, key: bytes) -> bytes:
    blocks = to_blocks(plain)
    out = bytearray()
    key = expand_key(key, len(blocks))
    for idx, block in enumerate(blocks):
        block = pad(block)
        assert len(block) == BLOCK_SIZE
        for _ in range(ROUNDS):
            block = enc_sub(block)
            block = enc_perm(block)
            block = bytearray(block)
            for i in range(len(block)):
                block[i] ^= key[idx]
        out.extend(block)
    return bytes(out)

if __name__ == '__main__':
    with open("flag", 'rb') as flag_file:
        flag = flag_file.read()
    with open("key", 'rb') as key_file:
        key = key_file.read()
    print(b64encode(encrypt(flag, key)).decode())

```

This seems like a custom AES implementation. Note that we are not provided a decryption routine, so let's simply write one.

Implementing one is not too complicated, one just needs to reverse the encrypt function step by step.

```
def decrypt(cipher: bytes, key: bytes) -> bytes:
    blocks = to_blocks(cipher)
    out = bytearray()
    key = expand_key(key, len(blocks))
    for idx, block in enumerate(blocks):
        for _ in range(ROUNDS):
            block = bytearray(block)
            for i in range(len(block)):
                block[i] ^= key[idx]
            block = dec_perm(block)
            block = dec_sub(block)
        out.extend(block)
    return bytes(out)
```

It's essentially the encrypt function in reverse, in encrypt, key is XORed at last in the for loop, we do it first.

Then we do reverse of permutation `dec_perm` and reverse of substitution `dec_sub` in the following functions.

For reversing `enc_sub`, we just need to find the index of corresponding byte in the `sbox`

```
def enc_sub(in_bytes: bytes) -> bytes:
    return bytes([sbox[b] for b in in_bytes])

def dec_sub(in_bytes: bytes) -> bytes:
    return bytes([sbox.index(b) for b in in_bytes])
```

To reverse the permutation,

```
def enc_perm(in_bytes: bytes) -> bytes:
    num = int.from_bytes(in_bytes, 'big')
    binary = bin(num)[2:].rjust(BLOCK_SIZE * 8, '0')
    permuted = ''.join([binary[pbox[i]] for i in range(BLOCK_SIZE * 8)])
    out = bytes([int(permuted[i:i + 8], 2) for i in range(0, BLOCK_SIZE * 8, 8)])
    return out

def dec_perm(in_bytes: bytes) -> bytes:
    out = bytearray(64)
    permuted = bin(int.from_bytes(in_bytes, 'big'))[2:].zfill(64) # just converting to binary
    for i in range(64):
        out[pbox[i]] = ord(permuted[i]) # should be ones and zeros but using ord as bytearrays are directly convertible to int
    out_bytes = int.to_bytes(int(out, 2), 8, byteorder='big') # converting to bytes again
    return out_bytes
```

Once we have decryption function set up, we can start exploring the challenge :)

The devil at work here is the `expand_key` function. One could easily verify that without using much brain :)

```
for i in range(20):
    print(expand_key(bytes([i]),8))
# 8 since we know the flag is 8*BLOCK_LENGTH bytes
#b'\x00\x01\x01\x01\x01\x01\x01\x01'
#b'\x01\x01\x01\x01\x01\x01\x01\x01'
#b'\x02\x10\x00\x00\x00\x00\x00\x00'
#b'\x03\xd9\xd1\xe1A\x81\x01\x01\x01'
#b'\x04\x00\x00\x00\x00\x00\x00\x00'
#b'\x05\xf9\xfa\xc1\x81\x01\x01\x01'
#b'\x06\x00\x00\x00\x00\x00\x00\x00'
#b'\x07QaA\x81\x01\x01\x01\x01'
#b'\x08\x00\x00\x00\x00\x00\x00\x00'
#b'\t\xd1\xa1A\x81\x01\x01\x01\x01'
#b'\n\x00\x00\x00\x00\x00\x00\x00\x00'
#b'\x0b\xe9\xbl!\xc1\x81\x01\x01\x01'
#b'\x0c\x00\x00\x00\x00\x00\x00\x00'
#b'\r\t\x11\xa1A\x81\x01\x01\x01'
#b'\x0e\x00\x00\x00\x00\x00\x00\x00'
#b'\x0f!\xc1\x81\x01\x01\x01\x01\x01'
#b'\x10\x00\x00\x00\x00\x00\x00\x00'
#b'\x11!A\x81\x01\x01\x01\x01\x01'
#b'\x12\x00\x00\x00\x00\x00\x00\x00'
#b'\x13y\x91aA\x81\x01\x01\x01'
```

Without even looking at the key expansion, one could say the keys it expands to are quite bad and possibly quite repetitive.

Voila, lets try randomly decrypting with a key.

```
flag_enc =
b64decode(b'hQWYogqLXU0+rePyWkNlBlaAX47/2dCeLFMLrmPKcYRLYZgFuqRC7EtwX4DRtG31XY4az+y0vJ
J/pwWR0/J9gg==')
print(decrypt(flag_enc, b'\x00'))
print(decrypt(flag_enc, b'\x01'))
#b'\xe2\xaa/\xb8}\xb2\xe1\x9d\xbe\xf0\xad\x1c\xe4)\xa77c3_is_4LW4YS_th3_4nsw3r(but_wit
h_optimiz4ti0ns)}'
#b'\x15\xa84N\xff\x83\x00{\xbe\xf0\xad\x1c\xe4)\xa77c3_is_4LW4YS_th3_4nsw3r(but_with_0
ptimiz4ti0ns)}'
```

One could already read a lot of the flag! We only lack the first two blocks of the flag.

Why can we read the rest of the flag by decrypting with some non-sense key?

Since in encrypt function each block is xored with the key byte at the corresponding position, we luckily end up encrypting it with byte `b'\x01'` for the last 6 bytes.

And xoring with `b'\x01'` would be the same byte again hehe.

But wouldnt it be lost amidst all the permutation and substitution??

No, since we are exactly reversing the permutation and substitution since the xor part dies out!

Why do we have so many 0's and 1's in the expanded key?

It is evident from this part

```
for num in [key[i % len(key)] * 2 for i in range(key_len)]:  
    cur = pow(cur, num, 256)  
    expanded.append(cur)
```

As cur is repeatedly raised to the power num, once cur hits 0 or 1, it will stay 0 or 1 out of its misery.

So all we need to figure out is the first two bytes, which should be quite easy!

```
for i in range(65536):  
    key = long_to_bytes(i)  
    if (a := decrypt(flag_enc, key)).startswith(b'rgbCTF'):  
        print(a, key)
```

Ugly solution in [solve.py](#)

And boom! we have our flag

**rgbCTF-brut3`f0rc3`is`4LW4YS`th3`4nsw3r(but`with`Optimiz4ti0ns)`**

| Interactive controls are omitted from this PDF. See the canonical HTML version.